

Project B: Active Learning & GNNs

Submission Instructions

1. **Submission date:** July 23, 2026, at 22:55.
2. **Contact:** For administrative or professional questions, contact **Omer** at `omer.y@campus.technion.ac.il`. Questions posted on the forum are answered faster than email. Professional questions must be written in English.
3. **Team submission:** Pairs only, unless otherwise approved in advance.
4. **Download:**
 - Section A files
 - Section B files
5. **Submission file format:** Each team should submit a zip file named `<ID1_ID2>`, where `ID<i>` represents the ID number of each team member.
 - **Section A:** Submit only `strategy.py` and `video_link.txt`.
 - **Section B:** Submit only `gnn.py`.
6. **Code compliance:** Ensure that your code adheres to the provided template.
7. **Server:** It is recommended to use the assigned server. Please follow the instructions in `Students - How to work with your VM.pdf`.
8. **Installations:** If you struggle with package installations, notify us via the forum as soon as possible.
9. **Grading:** The grade of each section constitutes 60% (Section A) and 40% (Section B) of the overall Project B grade.
10. **Academic integrity:** Cheating, plagiarism, and copying code or reports from other students are strictly forbidden and will be handled according to the Technion academic regulations.

Section A: Active Learning (60%)

Introduction

Section A focuses on *active learning* for binary classification: predicting whether an employee will leave the company (**Left**) or stay (**Stayed**).

You are given a large pool of unlabeled employee records and a small initial labeled set. You may iteratively request labels through an annotation oracle, but each unique query consumes budget. Your goal is to maximize predictive performance while respecting all constraints.

The final classifier is a `RandomForestClassifier` provided by the framework. You may *not* replace it with another model.

Files

Download the Section A package from the link in the submission instructions above. The package contains a `Section A/student/` directory:

- `strategy.py` — **you implement this file** (the only code file you submit for Section A).
- `run.py` — fixed entry point; calls your `run_active_learning(seed)`.
- `utils.py` — fixed framework (oracle, training, evaluation helpers).
- `evaluation.py` — fixed self-evaluation script.
- `constants.yaml` — local configuration for self-evaluation; **do not submit**.
- `data/` — local development data:
 - `pool.csv` — unlabeled candidate pool (no `Attrition` column).
 - `splits_seed_{1,2,3}.pkl` — initial 500 labeled `Employee` IDs per seed.
 - `test_seed_{1,2,3}.pkl` — labeled local test sets for self-evaluation.
 - `feature_columns.pkl`, `.pool_labels.pkl` — used internally by `utils.py`; do not read directly.

Important: Final grading uses a *hidden* pool and test set (different from your local `data/`), together with the staff copy of `constants.yaml`.

Objective

Implement `run_active_learning(seed)` in `strategy.py`. Given a seed $s \in \{1, 2, 3\}$, your function must run your active learning procedure and return a trained `RandomForestClassifier`.

Problem Setup

- **Pool size:** $\sim 14,900$ employees.
- **Initial labeled set:** 500 samples per seed, provided free (does not consume oracle budget).
- **Oracle budget:** At most **5,000 unique** `Employee` IDs may be queried via `call_oracle` over all iterations combined.
- **Seeds:** Three seeds determine the initial labeled set and randomness. Each seed uses a different initial split and local test set.
- **Runtime:** Your code must finish within **60 seconds** per seed on the grading machine.

What You May and May Not Modify

- ✓ **Modify:** `strategy.py`. item[×] **Do not modify:** Any other file.
- ✓ **Submit:** `strategy.py`. item[×] **Do not submit:** Any other file.
- ✓ **Add:** Helper functions inside `strategy.py`.
- × **Do not train on test** `Employee` IDs. `train_model` will raise an error if violated.
- × **Do not use models other than the provided Random Forest.**

Import Restrictions

In `strategy.py`, you may import only from these top-level packages: `numpy`, `pandas`, `sklearn`, `scipy`, `collections`, `warnings`, `typing`, and `utils`. Additional imports will cause self-evaluation (and grading) to fail.

Provided Functions (`utils.py`)

`load_pool()` Returns the unlabeled pool as a `DataFrame` (features + `Employee` ID; no labels).

`load_initial_labeled(seed)` Returns the seed-specific initial 500 labeled rows (`Attrition` encoded as 0/1).

`call_oracle(sample_ids)` Request labels for a list of `Employee` IDs from the pool. Returns a labeled `DataFrame`. Enforces the 5,000 unique-ID budget.

`prepare_xy(df)` Converts a labeled `DataFrame` to $(X, y, \text{employee_ids})$ with consistent one-hot encoding.

```
train_model(X, y, employee_ids, seed)
    Trains the fixed Random Forest. Rejects test IDs; warns on duplicate training IDs.

evaluate_model(model, seed)
    Returns F1 for the Left class on the local test set (for debugging).

get_oracle_usage()
    Returns oracle budget usage statistics.
```

Implementation Notes

- You control **when**, **how many**, and **which** samples to query – subject only to the oracle budget and runtime limit.
- You may use pseudo-labeling or other training-set augmentation, as long as you do not exceed the oracle budget and do not violate other constraints.
- Duplicate **Employee** IDs in training data are allowed but will trigger a warning.
- Your function must **return** the trained model; the framework handles saving, loading, and final evaluation.

Self-Evaluation

From the **Section A/student/** directory, run:

```
python evaluation.py
```

This script will:

1. Validate imports in **strategy.py**.
2. Run your code for seeds 1, 2, 3 (resetting the oracle each time).
3. Save and reload each model, then compute F1(**Left**) on the local test sets.
4. Enforce the 60-second runtime limit per seed.
5. Print per-seed scores and the mean F1 over seeds.

Local scores are indicative only; **final grading uses hidden data**.

Constraints Summary

Constraint	Value
Classifier	Random Forest (fixed hyperparameters)
Initial labeled samples	500 per seed (free)
Max oracle queries (unique IDs)	5,000
Seeds	1, 2, 3
Max runtime per seed	60 seconds
Submitted code file	strategy.py

Scoring Policy

Section A accounts for **60% of the Project B grade**. Within Section A:

- **75%** — functional performance (active learning implementation)
- **25%** — video presentation

Equivalently, out of a 100-point Project B grade, Section A contributes 60 points: up to 45 from functional performance and up to 15 from the video.

Functional score (75% of Section A). Let $F1_s$ be the F1 score for the **Left** class ($y = 1$) on the **hidden** test set for seed $s \in \{1, 2, 3\}$, computed by course staff after inserting your `strategy.py` into the official template. Your team score is:

$$\overline{F1} = \frac{1}{3} \sum_{s=1}^3 F1_s.$$

The functional component is worth **45 points** (75% of Section A), split into:

- **Guaranteed portion:** $\frac{2}{3} \times 45 = 30$ points
- **Competitive portion:** $\frac{1}{3} \times 45 = 15$ points

Every team with $\overline{F1} \geq 0.55$ is guaranteed at least **30 points**.

Teams ranked 1st, 2nd, and 3rd by $\overline{F1}$ (among all teams with $\overline{F1} \geq 0.55$) receive the full functional score (**45 points**), plus a bonus toward Section A:

- **1st place:** 45 points + **5 bonus points** toward Section A
- **2nd place:** 45 points + **3 bonus points** toward Section A
- **3rd place:** 45 points + **1 bonus point** toward Section A

For teams ranked 4th and below (with $\overline{F1} \geq 0.55$):

$$\text{Functional} = 30 + \frac{\overline{F1} - 0.55}{\overline{F1}_{3\text{rd}} - 0.55} \times 15.$$

If $\overline{F1} < 0.55$:

$$\text{Functional} = \frac{\overline{F1}}{0.55} \times 30 \quad (\text{no competitive points}).$$

Video presentation (25% of Section A). Submit `video_link.txt` containing a single URL (YouTube, Google Drive, etc.).

Format: maximum **100 seconds** (1:40 minutes); at most **6 slides**; **both** team members must speak meaningfully.

Content focus: Unlike Project A, the emphasis is quantitative and process-oriented. We want to understand your work process through discussion and plots—what you tried, what worked, and what did not.

Rubric (15 points total):

- **20%** (3 pts) — Clarity of explanation
- **50%** (7.5 pts) — Empirical evaluation: critical discussion, clear plots with legends, self-interpreting figures
- **20%** (3 pts) — Both members participate meaningfully
- **10%** (1.5 pts) — Time management (within 100 seconds; penalties apply for exceeding the limit)

Section B: GNNs (40%)

Introduction

The Cora dataset is a widely used benchmark in graph-based machine learning. It consists of a citation network of scientific publications. Each publication is a node, and an edge exists between two nodes if one publication cites the other.

Features: There are 2,708 nodes. Each node is represented by a bag-of-words feature vector of the publication abstract (1,433 binary features).

Edges: There are 10,556 edges.

Labels: Each publication belongs to one of 7 research-area classes: Neural Networks, Rule Learning, Reinforcement Learning, Probabilistic Methods, Genetic Algorithms, Case Based, and Theory.

Files

Download the Section B package from the link in the submission instructions above. The package contains a `Section B/student/` directory:

- `gnn.py` — **you implement this file** (the only code file you submit for Section B).
- `run.py` — fixed entry point; trains and evaluates your model for one seed.
- `utils.py` — fixed framework (data loading, graph construction, evaluation helpers).
- `evaluation.py` — fixed self-evaluation script.
- `constants.yaml` — local configuration for self-evaluation; **do not submit**.
- `data/` — Cora artifacts:
 - `nodes.csv` — node features and metadata.
 - `edges.csv` — citation edges.
 - `subject_mapping.pkl` — maps subject names to integer labels.
 - `indices_dict_part2.pkl` — train/validation/test node-ID splits.

Objective

Complete a GraphSAGE-based node classification pipeline in `gnn.py`. Your code must:

- Build the graph tensors from the CSV/pickle files.
- Define and train a **GraphSAGE** model.
- Save the best validation checkpoint as `best_model.pt` during training.

Instructions

- Implement all TODO sections in `gnn.py`. **Do not modify any other file.**
- You must use the **GraphSAGE** architecture (`SAGEConv`).
- Train for **100 epochs**.
- You **cannot** use test indices for training.
- Your model is evaluated on **3 random seeds** (1, 2, 3), which control initialization and training randomness.
- There are **no constraints on hyperparameters** (e.g., hidden dimensions, number of layers, learning rate, optimizer, dropout); choose them as you see fit.
- To receive full credit, test accuracy for each seed must be at least **0.79**.
- It is recommended (but not mandatory) to use the assigned GPU server for Section B.

What You May and May Not Modify

- ✓ **Modify:** `gnn.py` only.
- ✓ **Add:** Helper functions inside `gnn.py`.
- × **Do not modify:** `utils.py`, `run.py`, `evaluation.py`.
- × **Do not submit:** `constants.yaml`.
- × **Do not train on test node IDs.**

Import Restrictions

In `gnn.py`, you may import only from these top-level packages: `torch`, `torch_geometric`, `pandas`, `numpy`, `pickle`, and `typing`. Additional imports will cause self-evaluation (and grading) to fail.

Functions to Implement (`gnn.py`)

`get_feature_vectors(nodes_df)`

Return node features `x` as a float `torch.Tensor`.

`get_edges(edges_df, inverse_node_id_mapping)`

Return `edge_index` as a long tensor of shape `[2, num_edges]`.

`get_labels(nodes_df, subject_mapping)`

Return labels `y` as a long `torch.Tensor`.

`GraphSAGE.__init__`

Build the GraphSAGE layers. The framework instantiates `GraphSAGE(data.x.shape[1], output_dim, seed)`.

`GraphSAGE.forward`

Forward pass using `SAGEConv`.

`train(...)` Train for the requested number of epochs, track validation accuracy via the provided `evaluate_fn`, and save the best model to `best_model.pt`.

Self-Evaluation

From the `Section B/student/` directory, run:

```
python evaluation.py
```

This script will:

1. Validate imports in `gnn.py`.
2. Run your code for seeds 1, 2, 3.
3. Report per-seed test accuracy and the Section B score using the penalty formula below.

Scoring Policy

Section B accounts for **40 points** of the Project B grade (40% of the total project).

For each seed $s \in \{1, 2, 3\}$, let acc_s be the test-set accuracy returned by the official framework after inserting your `gnn.py`.

To receive the full Section B score (**40 points**), every seed must satisfy $\text{acc}_s \geq 0.79$.

Otherwise, points are deducted using the same rule as Homework 2:

$$\text{Penalty} = \sum_{s=1}^3 \left\lfloor \frac{0.79 - \text{acc}_s}{0.01} \right\rfloor,$$

$$\text{Section B score} = \max(0, 40 - \text{Penalty}).$$

Constraints Summary

Constraint	Value
Architecture	GraphSAGE (<code>SAGEConv</code>)
Training epochs	100
Hyperparameters	No fixed constraints
Seeds	1, 2, 3
Full-credit accuracy (per seed)	≥ 0.79
Submitted code file	<code>gnn.py</code>